Systém pro správu skladu elektronických součástek

WMS – Elektronické součástky

Projektová dokumentace

Autoři: Tomáš Simandl
Tomáš Szetei
Lukáš Kvapil

Datum: 5. dubna 2026

Dokumentace systému pro skladovou evidenci elektronických součástek

Obsah

1 Datový model **2**

1.1 Formulace problému 2

1.2 Datový slovník 2

1.3 Diagram tříd 4

1.4 Stavový diagram 6

1.5 Activity diagram – Vytvoření pohybu 9

1.6 Sekvenční diagram – Vytvoření pohybu 10

1.7 Use Case 11

Kapitola 1

Datový model

1.1 Formulace problému

Cílem tohoto projektu je teoreticky navrhnout informační systém pro správu skladových součástek. Systém má umožňovat evidenci součástek, jejich kategorií, dodavatelů a skladových lokací. Dále má podporovat sledování pohybů součástek ve skladu, jako je naskladnění, vyskladnění nebo přesun mezi jednotlivými lokacemi.

Součástí systému je také možnost přiřazovat součástky k projektům a evidovat jejich množství potřebné pro daný projekt. Návrh systému je znázorněn pomocí UML class diagramu, který popisuje jednotlivé třídy, jejich atributy a vztahy mezi nimi.

1.2 Datový slovník

Entity tříd

- **Soucastka**: Skladová položka evidovaná v systému WMS s aktuálním množstvím na skladě. Součástka může nabývat stavů: *Dostupna* (volná k použití), *Rezervovana* (blokována pro projekt), *Vydana* (fyzicky vydána ze skladu) nebo *Odepsana* (trvale vyřazena z evidence).
- **Projekt**: Projekt, ke kterému jsou přiřazeny součástky pro sledování spotřeby.
- **SkladovyPohyb**: Abstraktní třída reprezentující pohyb součástek – příjem, výdej nebo přesun.
- **PrijemPohyb**: Konkrétní pohyb typu příjem – naskladnění součástek od dodavatele (dědí ze **SkladovyPohyb**).
- **VydejPohyb**: Konkrétní pohyb typu výdej – vyskladnění součástek ze skladu (dědí ze **SkladovyPohyb**). Atribut `duvod` popisuje účel výdeje, např. *použití v projektu*, *reklamace*, *interní spotřeba* nebo *oprava*.

- **PresunutiPohyb**: Konkrétní pohyb typu přesun – přemístnění součástek mezi lokáními ve skladu (dědí ze **SkladovyPohyb**).
- **Sklad**: Fyzický sklad rozdělený na lokace, ve kterých jsou uloženy součástky.
- **Lokace**: Konkrétní místo v skladu identifikované označením (např. A-01-03).
- **Uzivatel**: Osoba spravující součástky, pohyby, projekty a sklady v systému.

Atributy

Tabulka 1.1: Přehled atributů tříd

Entita	**Atributy**
Soucastka	id, nazev, mnozstvi, stav (Dostupna | Rezervovana | Vydana | Odepsana)
Projekt	id, nazev
SkladovyPohyb	id, datum, stav, typ (prijem | vydej | presun), mnozstvi
PrijemPohyb	dodavatel – vlastní; id, datum, stav, typ – zděděno
VydejPohyb	duvod – vlastní; id, datum, stav, typ – zděděno
PresunutiPohyb	zdrojLokace, cilLokace – vlastní; id, datum, stav, typ – zděděno
Sklad	id, nazev
Lokace	id, oznaceni
Uzivatel	id, jmeno, email

1.3 Diagram tříd

Diagram zachycuje strukturu systému WMS a vztahy mezi jeho hlavními třídami. Součástky jsou přiřazovány k projektům a fyzicky umístěny v lokacích skladu. Třída `Uzivatel` reprezentuje uživatelskou roli v systému. Vytvoření pohybu provádí voláním `SkladovyPohyb.vytvoritPohyb()`, evidenci součástky voláním `Soucastka.evidovatSoucastku()` a přiřazení součástky k projektu voláním `Projekt.priradSoucastku()`.

Diagram využívá **generalizaci**: abstraktní třída `SkladovyPohyb` je základem pro tři konkrétní podtřídy `PrijemPohyb` (naskladnění), `VydejPohyb` (vyskladnění) a `PresunutiPohyb` (přesun mezi lokacemi), které dědí společné atributy a přidávají vlastní specifické atributy.

Třída `SkladovyPohyb` deklaruje abstraktní metodu `vytvoritPohyb()`, kterou každá podtřída implementuje po svém:

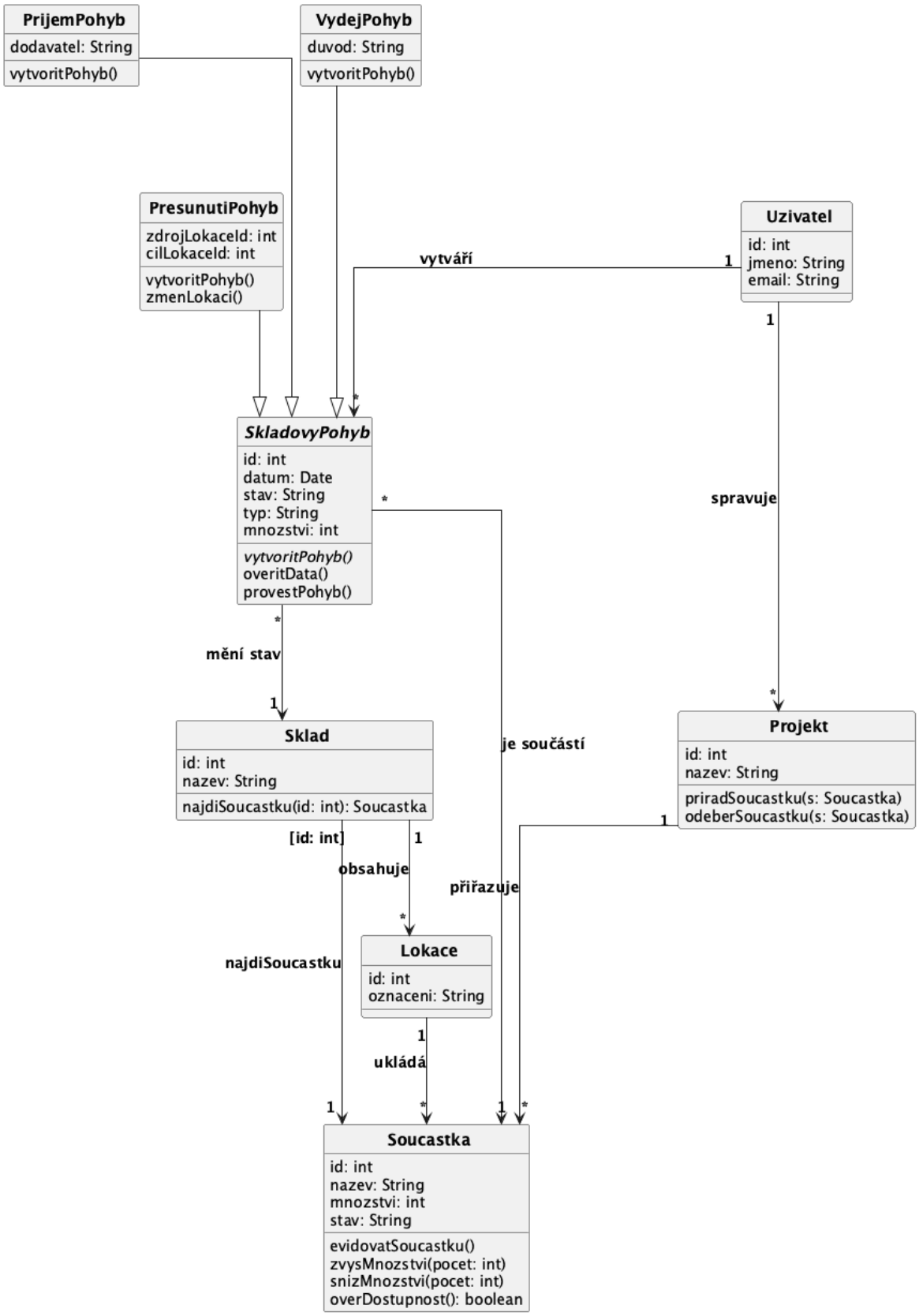
- `PrijemPohyb.vytvoritPohyb()` – vytvoří příjem a nastaví atribut `dodavatel`.
- `VydejPohyb.vytvoritPohyb()` – vytvoří výdej a nastaví atribut `duvod`.
- `PresunutiPohyb.vytvoritPohyb()` – vytvoří přesun a nastaví `zdrojLokaceId` a `cilLokaceId`.

Třída `SkladovyPohyb` zaznamenává každou změnu stavu zásob v systému. Konkrétně:

- `PrijemPohyb` – příjem součástek na sklad od dodavatele; zvyšuje množství dané součástky.
- `VydejPohyb` – výdej součástek ze skladu na projekt; snižuje množství dané součástky.
- `PresunutiPohyb` – přemístnění součástek mezi lokácemi ve skladu; mění přiřazenou lokaci součástky.

Každý pohyb se váže na konkrétní součástku (vazba `je součástí`, multiplicita $* \rightarrow 1$) a mění stav jednoho skladu (vazba `mění stav`, multiplicita $* \rightarrow 1$). Jeden uživatel může vytvořit libovolný počet pohybů (vazba `vytváří`, multiplicita $1 \rightarrow *$). Pohyb nemění fyzickou lokaci součástky – ta je pevně dána vazbou `Lokace` $\rightarrow$ `Soucastka`.

Vazba mezi `Sklad` a `Soucastka` je **kvalifikovaná** – sklad přistupuje ke konkrétní součástce prostřednictvím kvalifikátoru `id: int`.



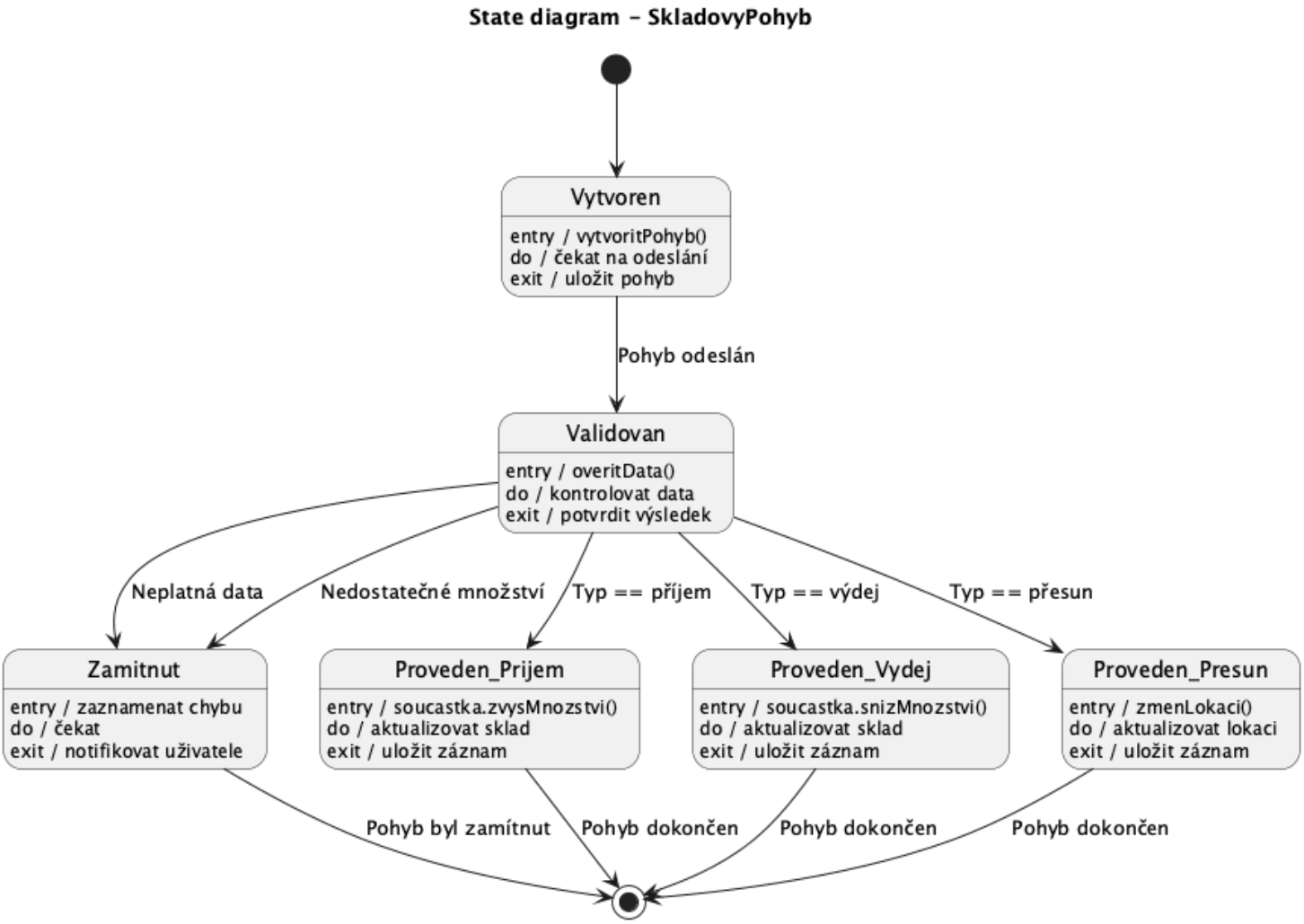
Obrázek 1.1: Class diagram WMS

1.4 Stavový diagram

Stavový diagram třídy `SkladovyPohyb`

Třída `SkladovyPohyb` reprezentuje obecný skladový pohyb – tedy jakoukoli změnu stavu zásob součástek v systému. Každý pohyb prochází životním cyklem řízeným třemi metodami:

- `vytvoritPohyb()` – inicializuje nový pohyb: přiřadí ID, nastaví datum vzniku, typ pohybu a vazbu na součástku;
- `overitData()` – ověří platnost zadaných dat: existenci součástky, vyplnění povinných polí, dostatečné množství na skladě (příjem/výdej) nebo platnost lokací (přesun);
- `provestPohyb()` – fyzicky provede pohyb: u příjmu zavolá `soucastka.zvysMnozstvi()`, u výdeje `soucastka.snizMnozstvi()` a u přesunu `zmenLokaci()`.



Obrázek 1.2: Stavový diagram třídy SkladovyPohyb

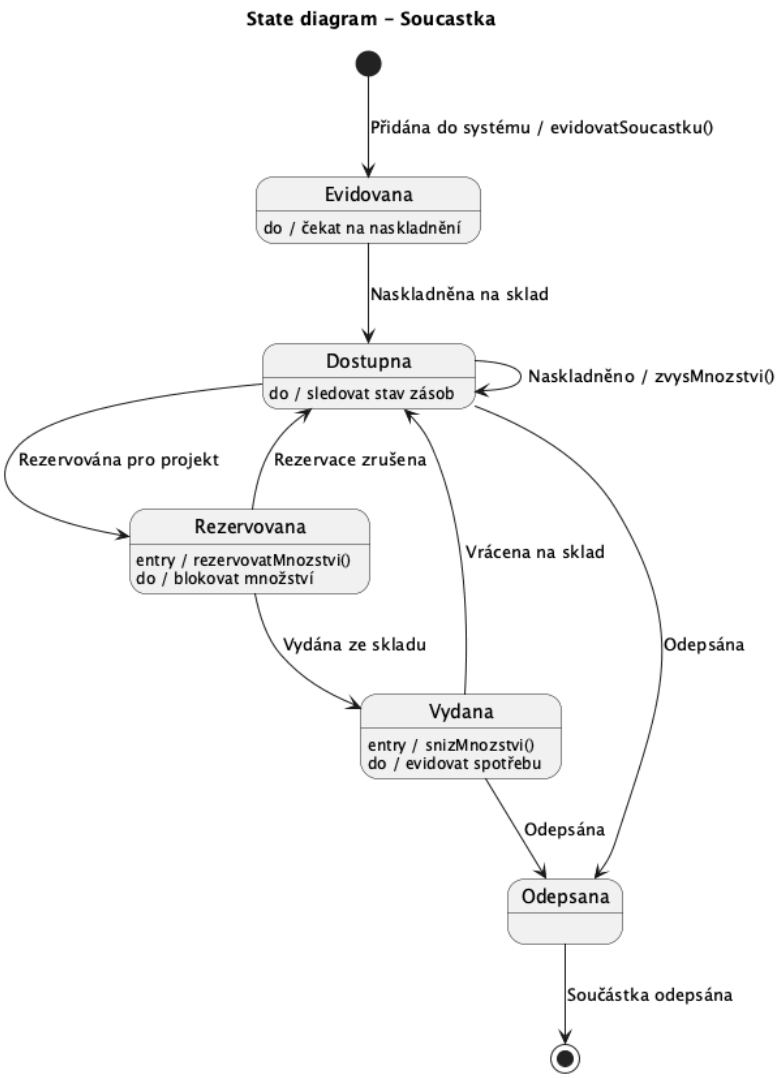
Stavový diagram třídy `Soucastka`

Diagram zachycuje, jak se mění stav součástky v průběhu skladových operací. Součástka je nejprve přidána do systému pomocí `evidovatSoucastku()` – nachází se ve stavu *Evidovana* s nulovým množstvím. Po naskladnění přechází do stavu *Dostupna*, kde je k dispozici pro rezervaci nebo výdej. Opakované naskladnění (např. doplnění zásob) ji nechá ve stavu *Dostupna* a zavolá `zvysMnozstvi()`.

Z tohoto stavu mohou nastat dva přechody:

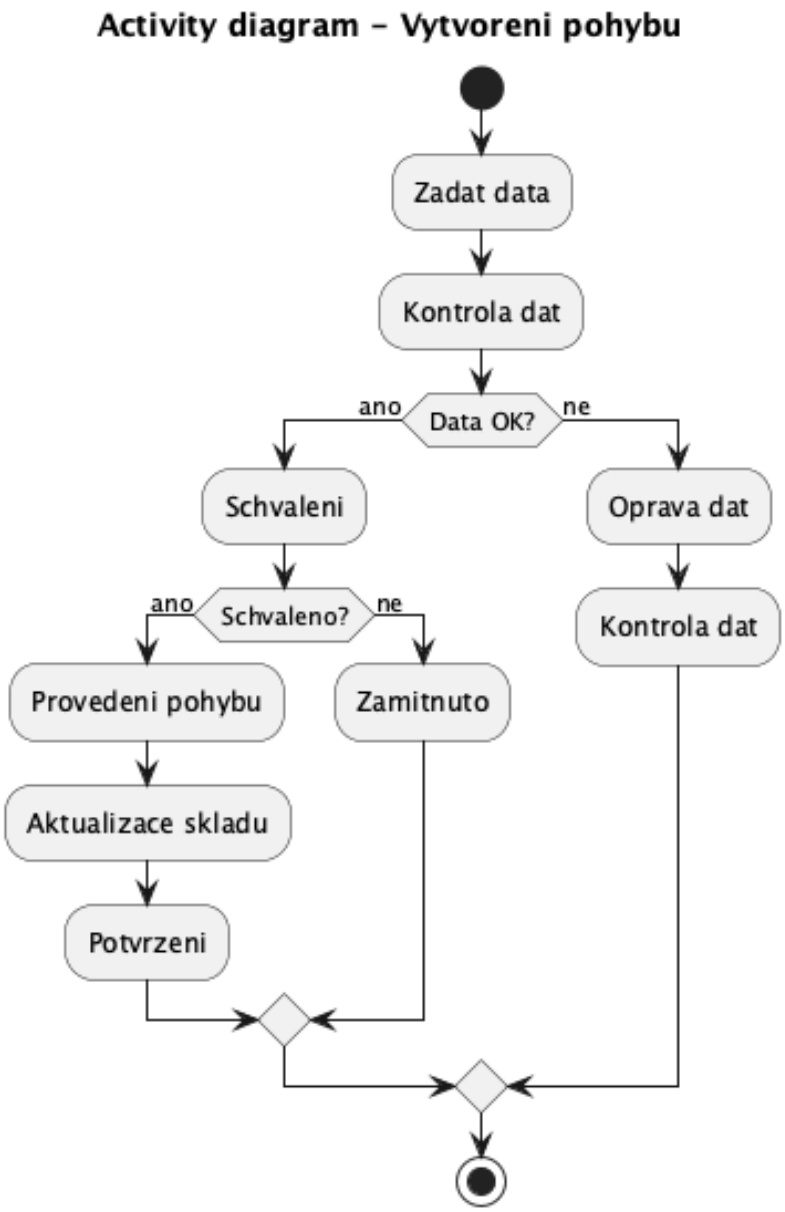
- **Rezervována pro projekt** – součástka přechází do stavu *Rezervovana*, kde je blokována pro konkrétní projekt (`rezervovatMnozstvi()`). Rezervaci lze zrušit a součástka se vrátí do *Dostupna*.
- **Vydána ze skladu** – součástka přechází do stavu *Vydana*, kde se zavolá `snizMnozstvi()`. Lze ji vrátit zpět do *Dostupna*, nebo odepsat.

Ze stavů *Rezervovana* i *Vydana* lze součástku **Odepsat** – přechází do terminálního stavu *Odepsana*, ze kterého již není návratu.



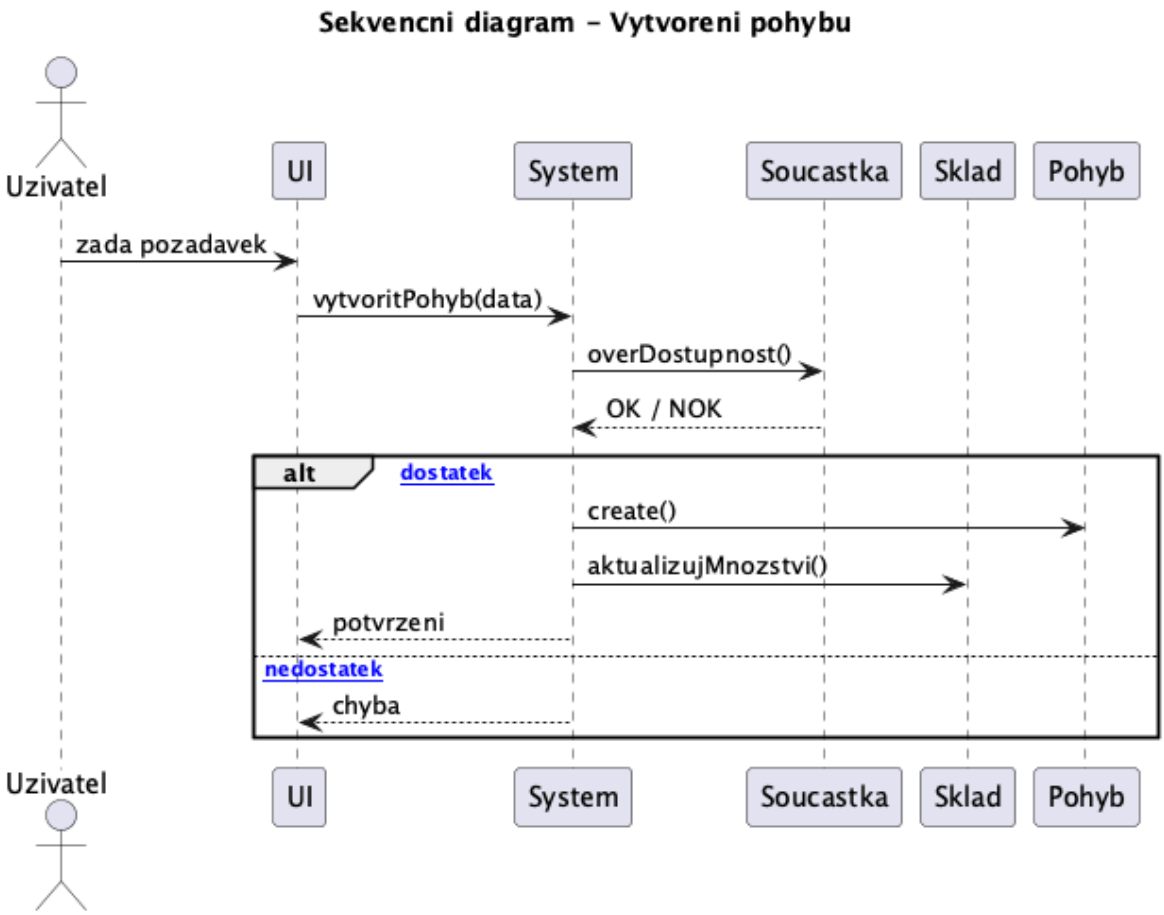
Obrázek 1.3: Stavový diagram třídy Soucastka

1.5 Activity diagram – Vytvoření pohybu



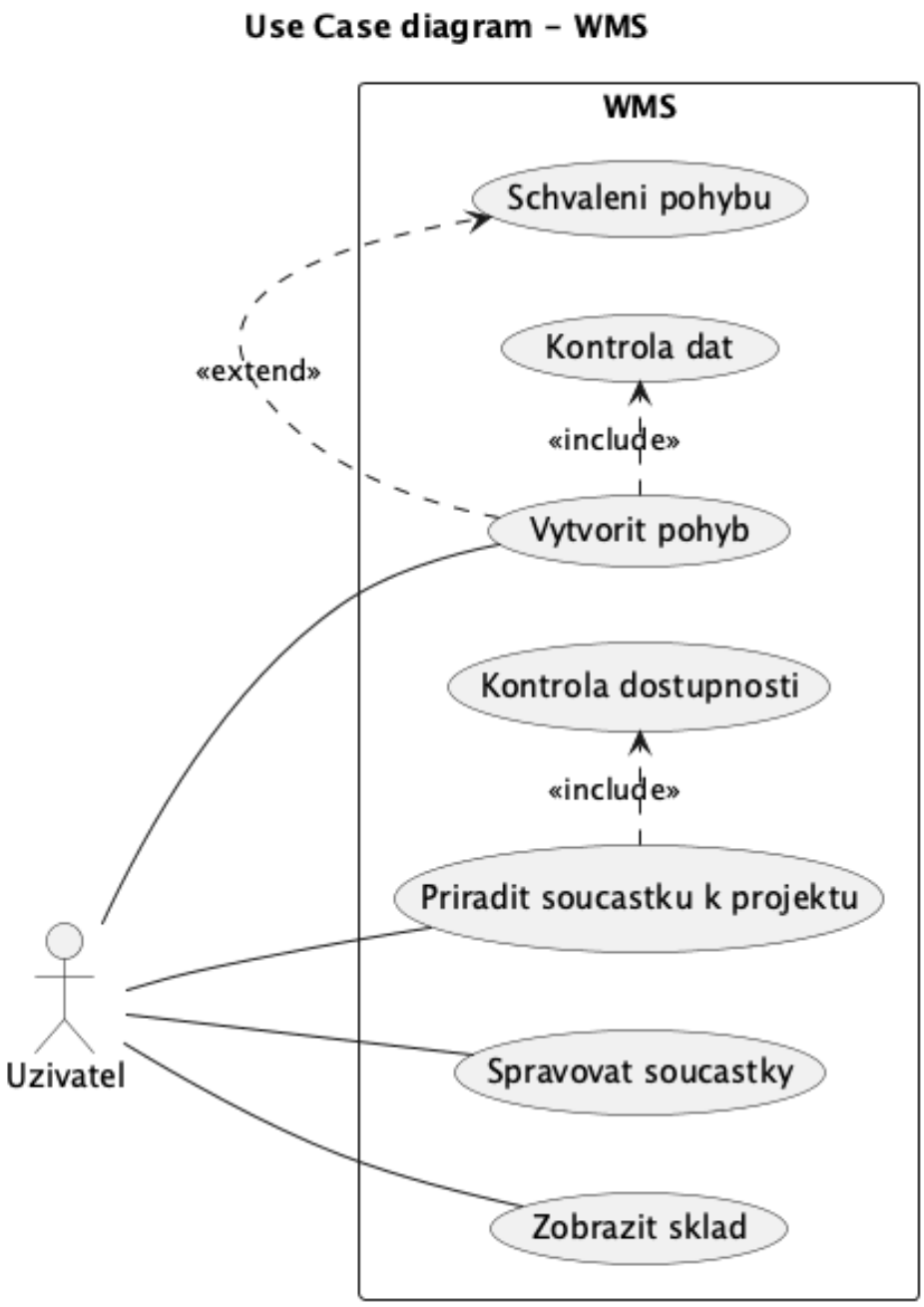
Obrázek 1.4: Activity diagram – Vytvoření pohybu

1.6 Sekvenční diagram – Vytvoření pohybu



Obrázek 1.5: Sekvenční diagram – Vytvoření pohybu

1.7 Use Case



Obrázek 1.6: Use Case Diagram – WMS

Scénář: Vytvoření pohybu

1. Uživatel zvolí *Vytvořit pohyb*.
2. Zadá součástku a množství.
3. Systém provede kontrolu dat.
4. Pokud `[data_ok]` → pohyb je schválen.

5. Systém provede pohyb a aktualizuje stav skladu.
6. Zobrazí se potvrzení výsledku.

Scénář: Přiřazení součástky k projektu

1. Uživatel vybere projekt.
2. Zvolí součástku a zadá množství.
3. Systém ověří dostupnost voláním `overDostupnost()`.
4. Pokud `[dostatek]` → součástka je přiřazena.
5. Systém aktualizuje množství na skladě.